1. INTRODUCERE – PREZENTARE GENERALĂ

1.1 Introducere

Managementul materiei prime are o importanță deosebită în operațiunile multor companii, fie că se referă la eficientizarea lanțurilor de producție sau la gestionarea întregului proces de aprovizionare. Totuși, administrarea manuală a acestor activități nu constituie, de cele mai multe ori, soluția optimă în ceea ce privește monitorizarea erorilor, stocurilor și a pierderilor de timp.

În prezent, opțiunea digitalizării proceselor de lucru dintr-un depozit a devenit o necesitate, fiind esențială îndeplinirii standardelor actuale de organizare și protecție a datelor. Adaptarea instrumentelor la spațiul electronic oferă o evidență precisă, limitează dezvoltarea erorilor și permit un control sporit asupra activităților efectuate.

Principiul de a extinde utilizarea acestor mijloace stă la baza realizării unui proiect menit să fluidizeze, atât procurarea de bunuri necesare, cât și coordonarea resurselor umane.

1.2 Prezentarea proiectului

Lucrarea urmărește realizarea unei aplicații informatice care să ofere sprijin în gestionarea stocurilor și vizualizarea clară a produselor, tranzacțiilor și utilizatorilor, păstrând totodată o interfață prietenoasă și eficientă. Platforma integrează un sistem de autentificare bazat pe mai multe roluri, având două categorii de utilizatori: operatori și administratori.

Operatorul, având un rol limitat, are posibilitatea de a examina produsele înregistrate în depozit, urmărind detaliile acestora, dar și calitatea de introducere a noilor tranzacții. În cazul editării mișcărilor de stoc, operatorul are dreptul doar de a modifica descrierea, fără a fi posibilă ștergerea sau a schimbarea lor semnificativă.

Administratorul, pe de altă parte, are un control complet asupra aplicației, fiindu-i atribuită producerea sau prefacerea oricărui element, procedeu sau client al platformei. Administrarea utilizatorilor este vizibilă exclusiv celor care au acest rol, subliniind distincția clară dintre cele două niveluri de acces.

1.3 Beneficiile informatizării procesului de gestionare a materiilor prime

Metoda clasică de management a materiilor prime constituie un efort semnificativ în ceea ce privește urmărirea inventarului și actualizarea tranzacțiilor, existând posibilitatea expunerii lor la erori și lipsă de ordine. Datorită integrării fluxurilor de lucru într-un mediu digital, ne putem bucura de o multitudine de beneficii însemnate, care ne permit să monitorizăm eficient și organizat fiecare proces în parte. Indiferent dacă ne referim la actualizarea constantă a tuturor produselor, notelor de intrare și respectiv ieșire sau la personalul înregistrat în sistem, performanța se resimte atât în materie de timp cât și în materie de securitate și integritate a datelor.

Prin urmare, dirijarea complexă a procedurilor fundamentale oricărui depozit de materie primă, răspunde unei nevoi concrete din câmpul economic, dar și a cerinței dezvoltărilor viitoare

1.4 Stadiul actual

În prezent există o varietate de aplicații web care permit îndeplinirea numeroaselor nevoi specifice, ce includ urmărirea loturilor, vizualizarea activităților, gestionarea mai multor depozite și generarea de rapoarte detaliate. Câteva exemple relevante sunt prezentate în cele ce urmează.

1.4.1 InvenTree

InvenTree[1] este o platformă de planificare și control a stocurilor cu o multitudine de caracteristici diferite. Concentrată atât pe furnizori și producători, cât și pe vânzările către clienți, reușind astfel să urmărească tot ce ține de lanțul de aprovizionare. Ceea ce este unic în această interfață este reprezentat de integrarea API-urilor care facilitează legătura cu alte aplicații externe.

Pagina de start include și o variantă demo concepută pentru evidențierea modului în care sistemul de roluri și permisiuni funcționează. Nivelurile diferite de acces, conferă o experiență unică datorită perspectivei variate a fiecărui punct de autorizare în parte, facilitând înțelegerea avantajelor unui sistem flexibil în administrarea unui depozit digitalizat.

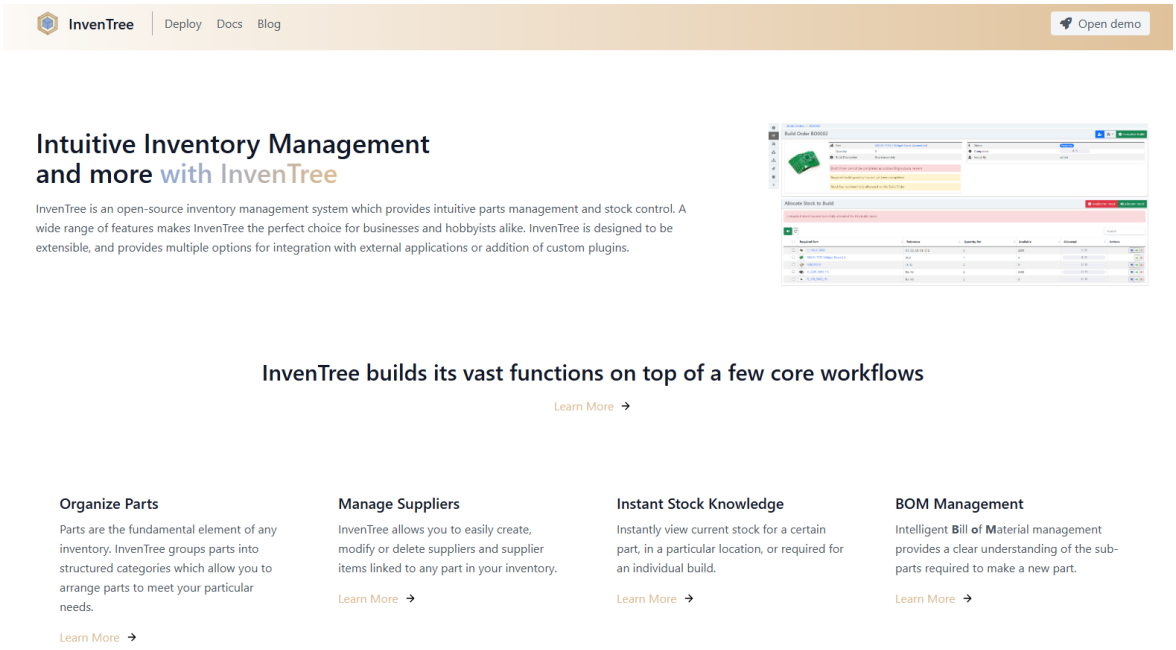


Fig.1.1 Pagina de start InvenTree[1]

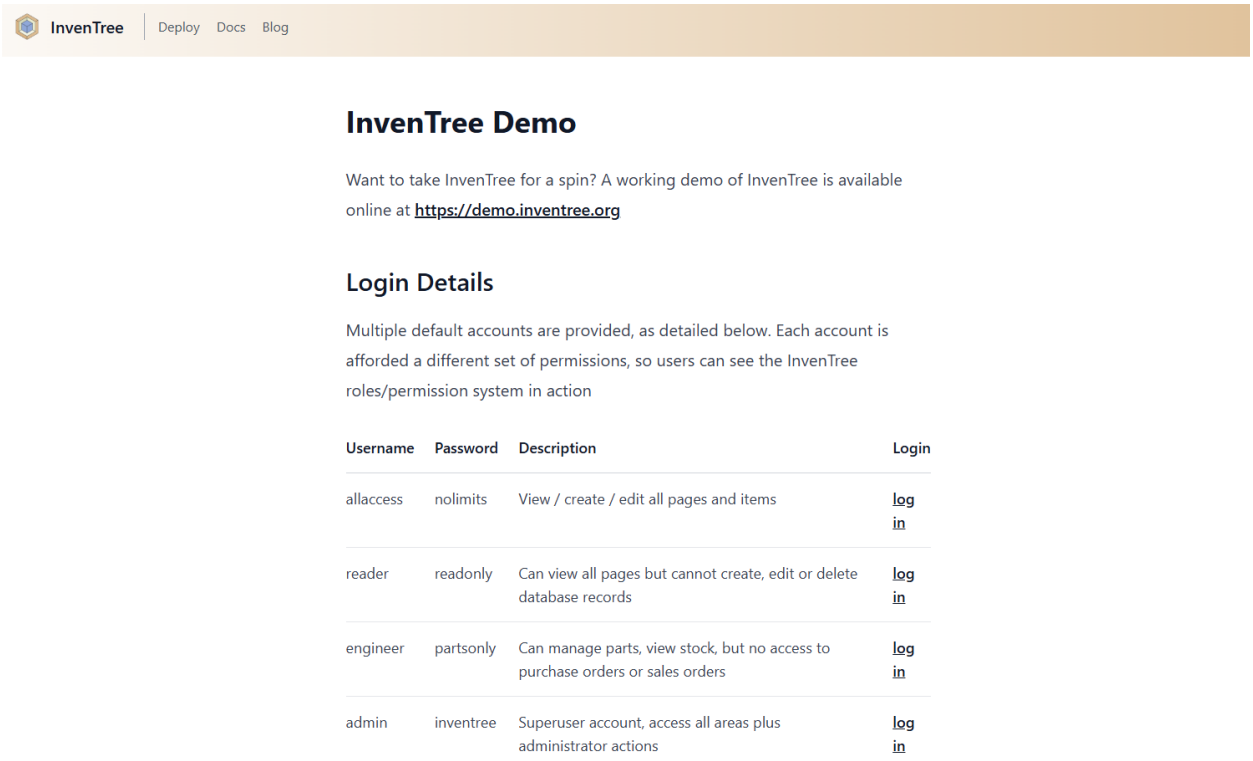


Fig.1.2 Pagina demo InvenTree[1]

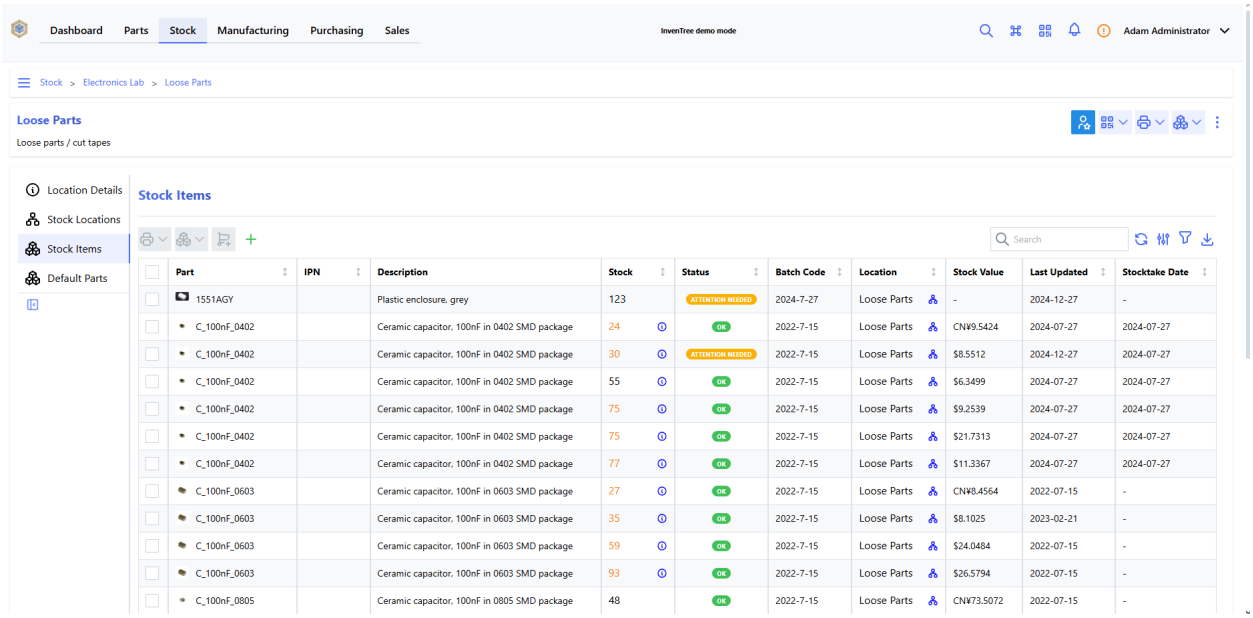


Fig.1.3 Vizualizarea stocului în InvenTree[1]

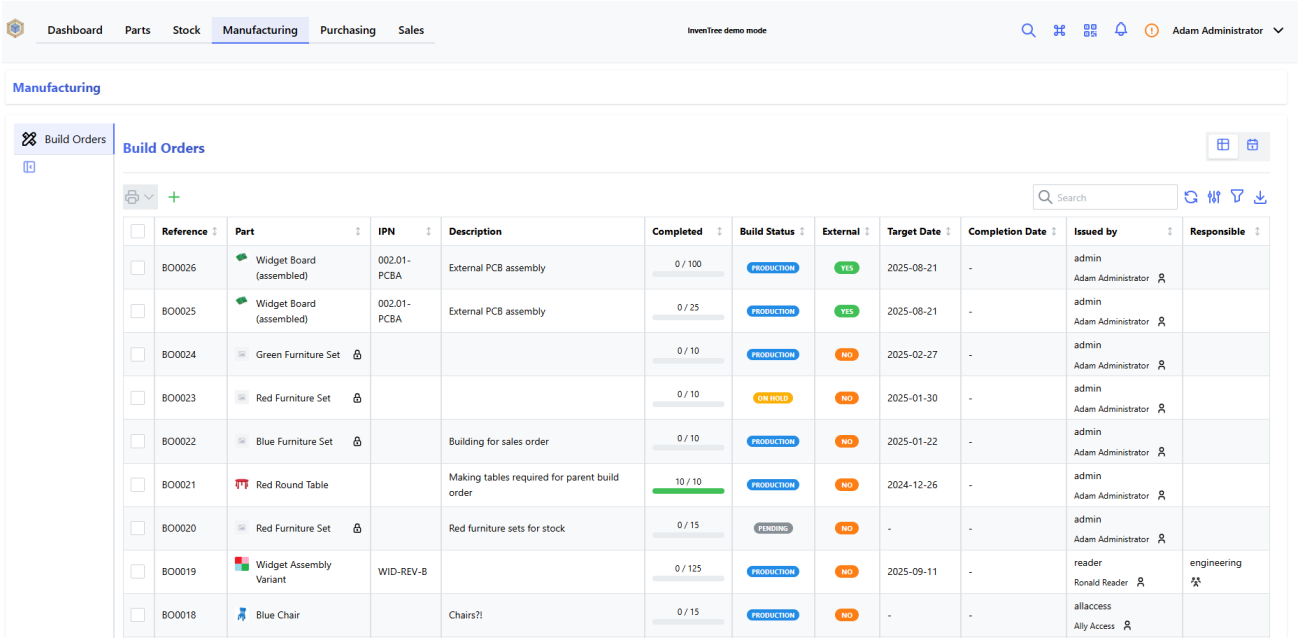


Fig.1.4 Vizualizarea producției în InvenTree[1]

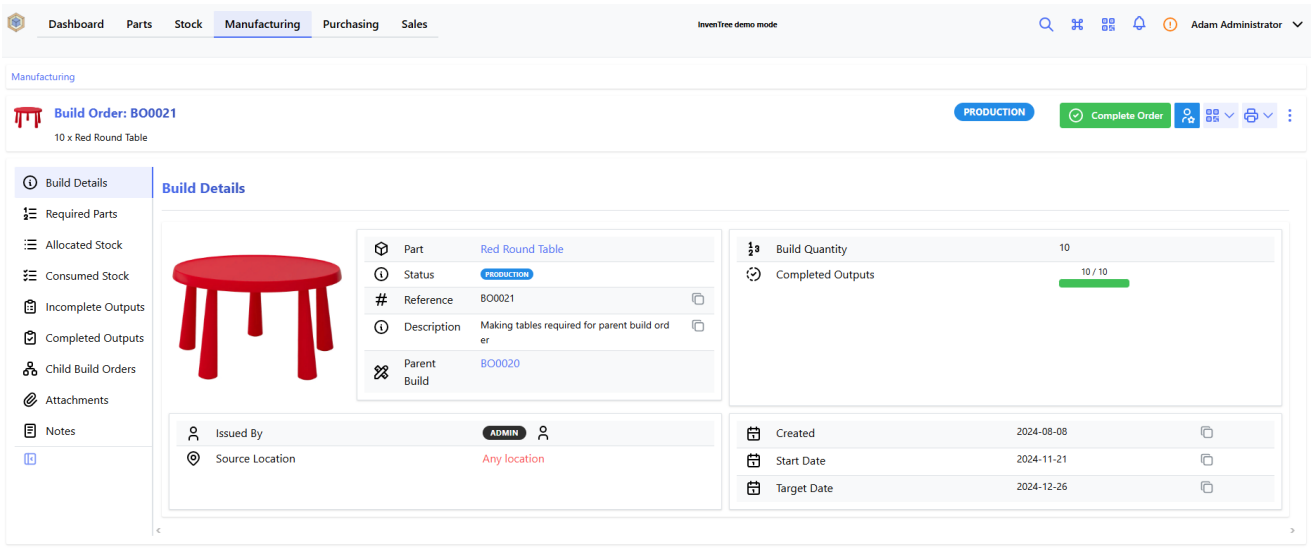


Fig.1.5 Vizualizarea ordinelor de asamblare în InvenTree[1]

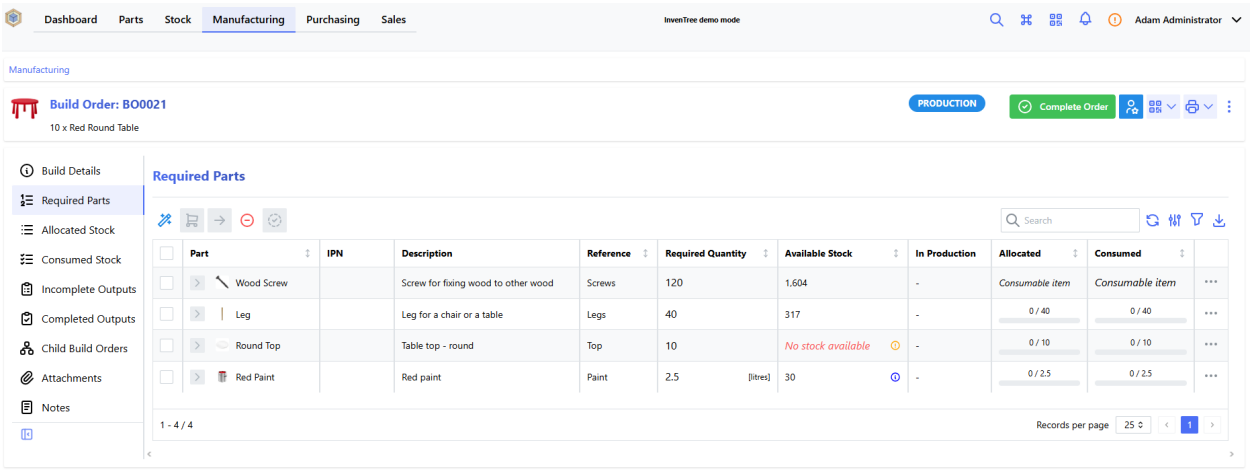


Fig.1.6 Vizualizarea componentelor necesare asamblării în InvenTree[1]

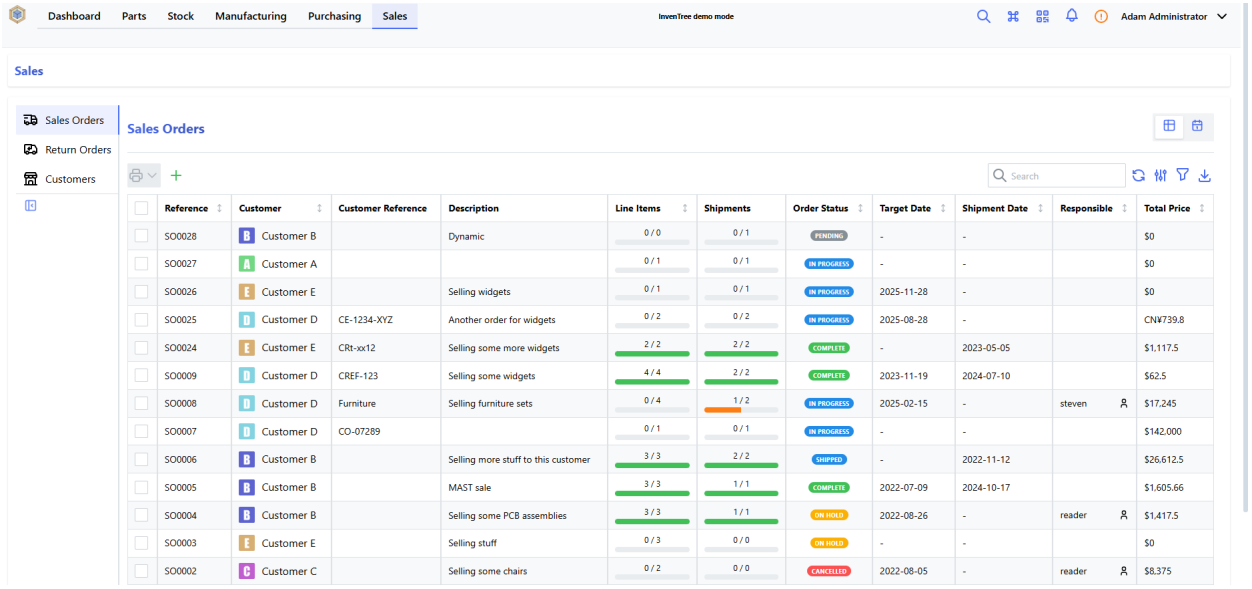


Fig.1.7 Vizualizarea clienților în InvenTree[1]

1.4.2 Odoo

Odoo[2] este o tehnologie ce permite utilizatorilor să își monitorizeze activitatea fără a folosi programe separate pentru contabilitate, vânzări, gestionare de produse sau resurse umane, toate acestea aflându-se într-un singur loc și folosind aceeași bază de date. Cu un design intuitiv, utilizatorul își poate alege doar programele de care are nevoie, iar în funcție de ele poate asigura un mediu de lucru simplu și eficient.

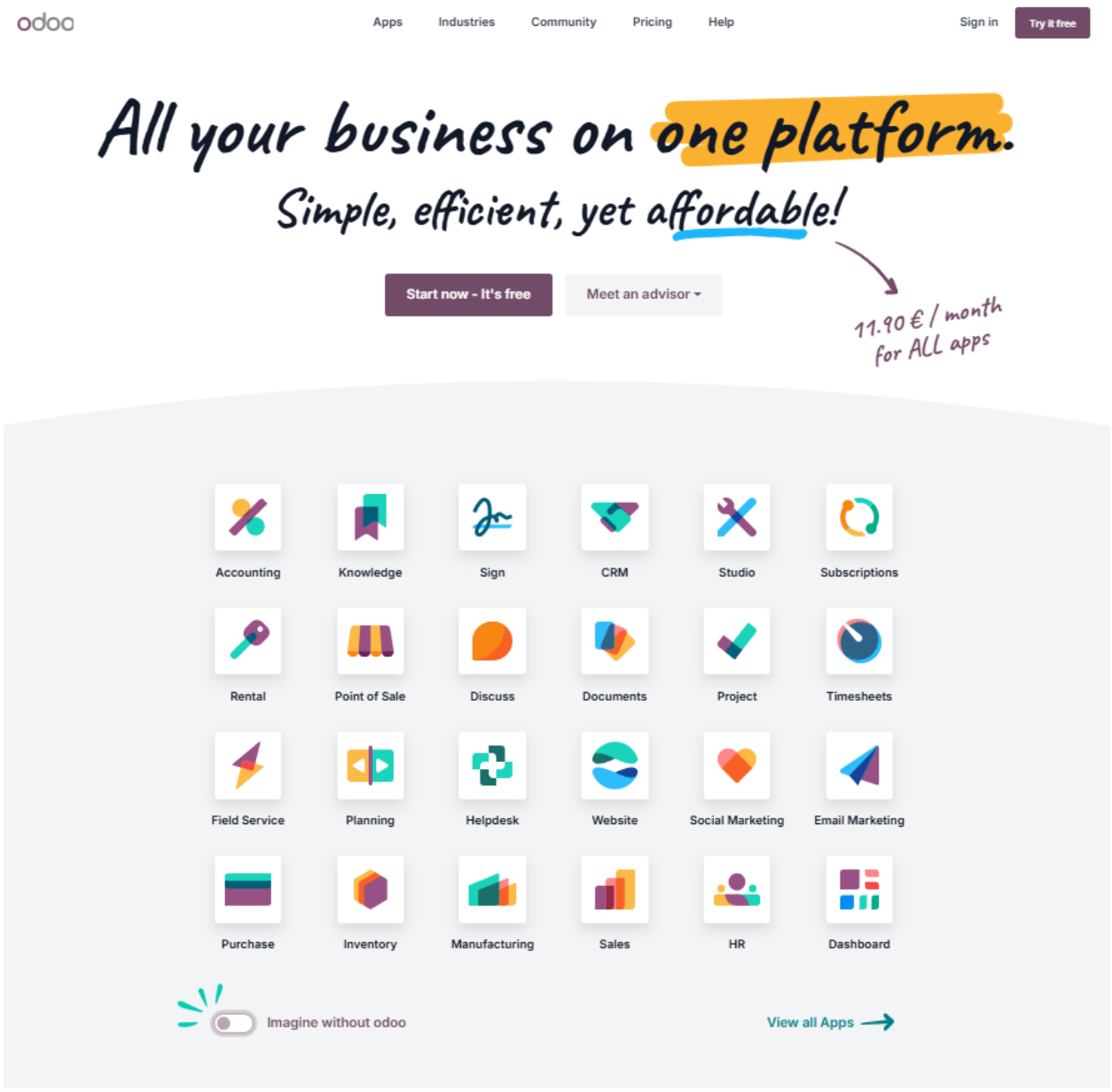


Fig.1.8 Pagina de start Odoo[2]

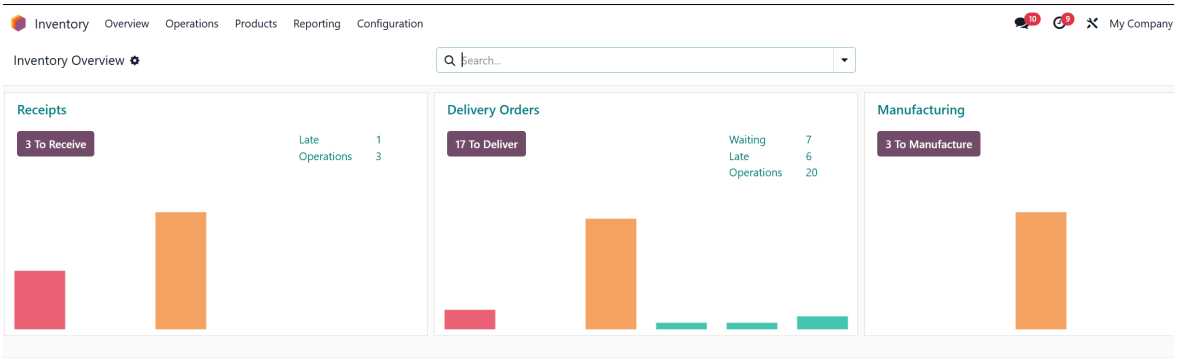


Fig.1.9 Prezentarea generală a inventarului în Odoo[2]

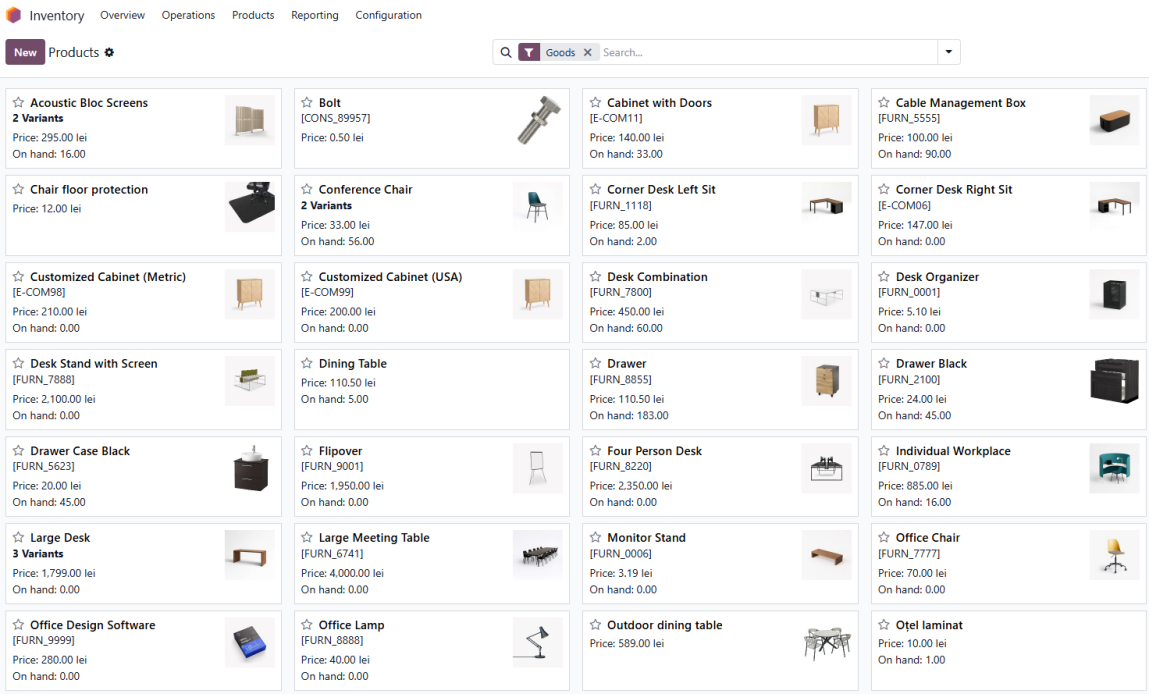


Fig.1.10 Afișarea produselor în Odoo[2]

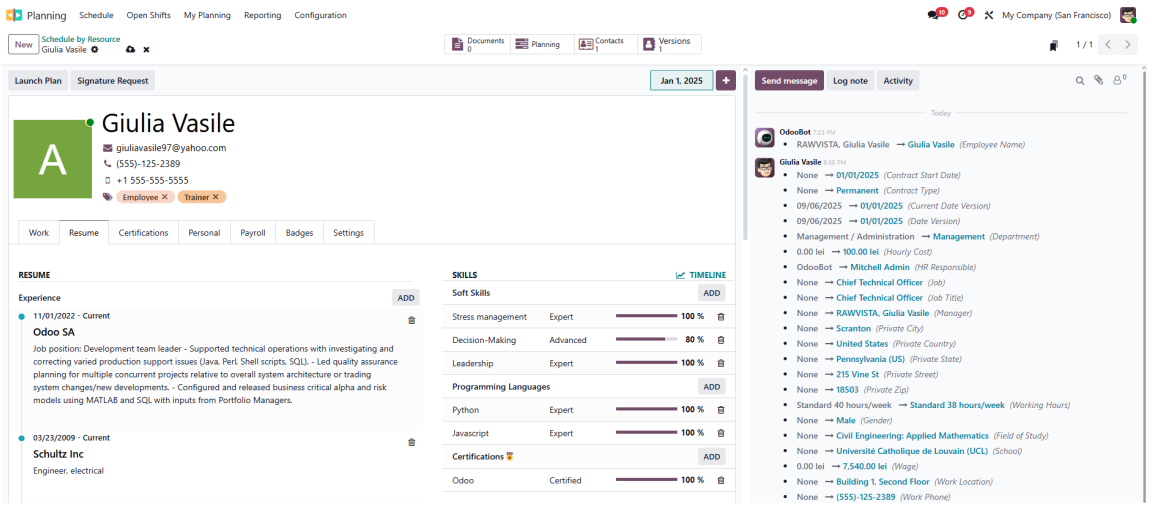


Fig.1.11 Afișarea datelor personalului în Odoo[2]

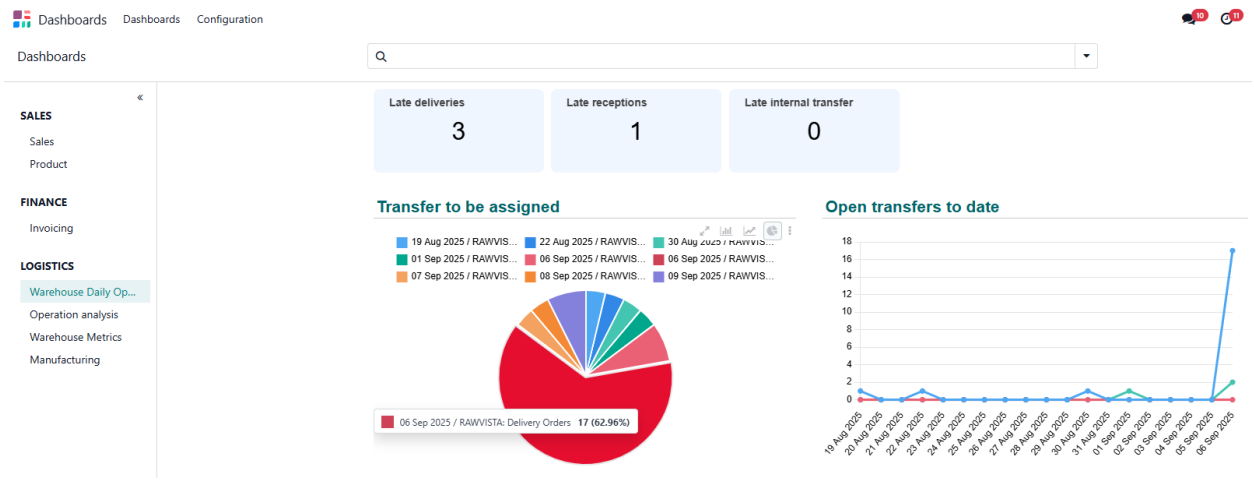


Fig.1.12 Dashboards în Odoo[2]

2. PRINCIPII ȘI METODE UTILIZATE PENTRU REZOLVAREA TEMEI DE PROIECT

2.1 Tehnologii utilizate

2.1.1 JavaScript – Limbaj de programare

JavaScript[3] este un limbaj de programare, interpretat, utilizat special pentru dezvoltarea aplicațiilor web. Fiind un limbaj ușor de învățat și flexibil, are o dinamică care permite implementarea unor funcții interactive în paginile noastre web.

Programarea bazată pe evenimente permite manipularea DOM-urilor (Modelul de obiecte al documentului), oferind dezvoltatorilor capacitatea de a modifica în timp real structura și stilul unei pagini. Facilitatea în realizarea aplicațiilor complexe este evidențiată de numeroase biblioteci și framework-uri care permit folosirea JavaScript atât pe partea de backend, cât și pe cea a clientului.

În plus, JavaScript este unul dintre cele mai folosite limbaje de programare, fiind esențial progresului digital. Totodată dispune de actualizări în mod regulat, ajutând astfel la îmbunătățirea funcționalității și performanței.

2.1.2 Node.js – Mediu de execuție pentru JavaScript pe partea de server

Node.js[4] este un mediu de execuție pentru JavaScript care rulează pe motorul JavaScript V8, în afara browser-ului, ceea ce îi permite să fie foarte performant în dezvoltarea aplicațiilor server-side.

Cu un model asincron orientat pe evenimente, reușește să administreze mii de conexiuni simultane cu un singur server, optimizând astfel utilizarea resurselor. De asemenea, Node.js include unul dintre cele mai mari registre de pachete și module reutilizabile care permit extinderea accelerată a oricărui proces în dezvoltare.

În prezent, Node.js este una dintre cele mai folosite tehnologii pentru realizarea oricărei platforme moderne, justificarea rămânând flexibilitatea și comunitatea care prioritizează performanța.

2.1.3 Express – Framework backend

Express[5] este cel mai utilizat framework bazat pe Node.js și servește drept bibliotecă principală pentru diverse alte framework-uri populare de Node.js. Conceput pentru o gestionare flexibilă a rutelor și cererilor HTTP unde arhitectura sa modulară simplifică procesul de creare a serverelor.

De asemenea, introducerea de „middleware” tratează cerelile în orice etapă a procesului de manevrare a acestora, ele fiind extinse rapid cu funcționalități precum autentificare, logare, validarea datelor sau tratarea erorilor.

Succesul de care dă dovadă Express este reprezentat de reducerea timpului necesar implementării soluțiilor complexe, atât pentru dezvoltatorii începători, cât și pe cei avansați. Așadar, Express rămâne una dintre cele mai utilizate soluții backend, construind baza multor aplicații web moderne.

2.1.4 PostgreSQL – Sistem de gestiune a bazelor de date

PostgreSQL este un sistem important de baze de date relaționale, open source, care utilizează și extinde limbajul SQL oferind astfel siguranță în rezolvarea celor mai complicate sarcini de lucru. Reputația solidă este câștigată prin arhitectura sa fiabilă și integritatea datelor care oferă în mod constant soluții performante și inovatoare.

Unul dintre principalele avantaje ale PostgreSQL constă în compatibilitatea cu tranzacțiile ACID (Atomicitate, Consistență, Izolare, Durabilitate) definite de utilizator. Adăugarea de funcții personalizate, integritatea datelor și mecanismele eficiente de optimizare a interogărilor, justifică preferința dezvoltatorilor și a marilor companii de a alege această tehnologie.

Datorită arhitecturii robuste, a comunității open-source și a extensibilității de care dă dovadă, PostgreSQL constituie alegerea frecventă în diverse proiecte indiferent de cât de mare sau mic este setul de date.

2.1.5 pg – Driver PostgreSQL pentru Node.js

pg[6] este un modul oficial Node.js care permite conectarea cu baza de date din PostgreSQL. Acesta include mecanisme de administrare a erorilor datorită suportului pentru interogări parametrizate, devenind astfel standardul principal de conectare dintre PostgreSQL și Node.js.

2.1.6 HTML – Hyper Text Markup Language

HTML (Hyper Text Markup Language)[7], [8] este un limbaj marker standard pentru crearea paginilior Web. Prin intermediul acestuia, se pot crea și organiza elemente de tip text, titlu, paragrafe, imagini, link, listă, tabel cu ajutorul etichetelor de tip tags, proces care se numește marcare. Etichetele au nume sugestiv și sunt încadrate între simbolurile ”<” ”>”. Acestea nu sunt case-sensitive, deci pot fi scrise cu litere mici sau majuscule, fără a fi afectată funcționalitatea. Etichetele sunt poziționate la începutul și sfârșitul unei secțiuni, cum ar fi <title> Title </title>.

Secțiunea head a unui document HTML nu apare pe site-ul propriu-zis atunci când pagina este încărcată, conținând informații precum: titlul (<title>), link-uri către CSS, sau alte elemente utilizate pentru stilizarea paginii web și date despre documentul HTML.

Secțiunea body a unui document HTML include elemente de tip heading (<heading>), paragraf (<p>), liste (<li>), formulare (<form>), care formează conținutul principal al paginii web. Acestea pot fi așezate în pagină cu ajutorul containerelor de tiv , elemente block non-semantice care se ocupa cu organizarea informațiilor din pagina în secțiuni separate.

2.1.7 CSS – Cascading Style Sheets

CSS (Cascading Style Sheets) [9] oferă posibilitatea de înfrumusețare a paginilor web, oferind control asupra modului în care acestea sunt prezentate către utilizator. În general, paginile web sunt scrise în limbaje de marcare precum HTML sau XML care apar în browser ca niște documente text, fără o structură, culori sau stiluri de afișare specifice.

CSS este un limbaj bazat pe reguli, ceea ce înseamnă că dezvoltatorul poate crea reguli specificând grupuri de stiluri care vor fi aplicate către anumite elemente ale paginii Web. Acestea sunt definite prin intermediul unui sistem de tipul proprietate-valoare. Se pot aplica mai multe proprietăți în cadrul unui element, care însumate, vor rezulta în stilul de afișare dorit.

2.1.8 Bootstrap – Framework pentru frontend

Bootstrap[10] este un toolkit pentru frontend robust și extensibil care permite personalizarea ușoară și rapidă a interfețelor Web folosind SASS (Syntactically Awesome Style Sheets), un sistem de așezare în pagină bazat pe o grilă de plugin-uri JavaScript ce pot fi incluse cu ușurință în proiecte pentru a oferi un aspect plăcut și pentru a genera elemente interactive în cadrul paginii Web, într-un timp scurt.

Printre clasele și elementele disponibile, se numără următoarele: butoane, formulare, elemente de tip carusel, dropdown-uri, bară de navigare, card-uri, modale, alerte și elemente ce țin de așezarea în pagină, reprezentate de sistemul grilă, împărțit în linii și coloane.

Bootstrap poate fi instalat prin intermediul npm (Node Package Manager), folosinf comenzile ”npm install bootstrap@5.3.3”.

2.1.9 React – Bibliotecă frontend

React[11],[12] este o bibliotecă JavaScript pentru randarea interfețelor utilizatorilor (UI) care este construită din unități mici, cum ar fi butoane, text și imagini. Indiferent de aplicația la care dezvoltatul lucrează, React îi permite combinarea lor în componente reutilizabile.

Documentația extinsă și biblioteca de resurse contribuie la evoluția și popularitatea sa continuă deoarece poate fi integrat cu alte pachete sau framework-uri pentru gestionarea și dezvoltarea de platforme moderne.

De asemenea, React și ReactDOM sunt atât de des utilizate împreună pentru că au roluri complementare care creează și optimizează sarcinile proiectelor. Abordarea fiecărui framework JavaScript oferă o modalitate de a scrie interfețe într-un mod mai declarativ, descrierea lor fiind posibilă în DOM care face ca acest lucru să fie posibil.

2.1.10 Vite – Instrument în dezvoltarea frontend

Vite[13] este un instrument de dezvoltare mai rapidă și mai eficientă pentru proiectele web moderne. A fost dezvoltat de Evan You, cunoscut și ca autor al framework-ului Vue.js, și s-a răspândit rapid datorită simplității și performanței sale.

Principalul avantaj al Vite este folosirea modulelor ESM (ECMAScript), pe care browser-ul le poate încărca rapid, unde transformă și servește codul sursă așa cum este solicitat. Codul din spatele importurilor dinamice condiționate este procesat numai dacă este utilizat efectiv pe ecranul curent.

Datorită comunității în creștere și a documentației accesibile, Vite a devenit una dintre cele mai populare soluții moderne pentru construirea interfețelor web, fiind totodată și foarte flexibil deoarece permite integrarea cu mai multe framework-uri populare precum React.

2.1.11 Axios – Bibliotecă pentru cereri HTTP

Axios[14] este o bibliotică JavaScript open-source popular, utilizată pentru a face cereri HTTP din browsere web sau medii Node.js. Este o modalitate de a simplifica trimiterea cererilor asincrone către endpoint-uri REST, gestionarea răspunsurilor și efectuarea diverselor sarcini legate de rețea.

Construit pe o bază de XMLHttpRequest din JavaScript și a API-ului fetch, Axios oferă o API mai ușor de utilizat, cu funcții precum transformarea automată a datelor JSON, gestionarea erorilor și suport pentru browserele mai vechi.

Datorită sintaxei simplificate, consistenței datelor, compatibilității și a securității îmbunătățite, Axios a devenit una dintre cele mai utilizate biblioteci pentru cereri HTTP în proiectele bazate pe utilizarea framework-urilor precum React sau a programelor realizate în JavaScript.

2.1.12 SheetJS (xlsx) – Bibliotecă pentru manipularea fișierelor Excel

SheetJS [15] este biblioteca JavaScript care permite analizarea și scrierea în diverse formate de foi de calcul, inclusiv xlsx din Excel. Devine foarte utilă în proiectele unde datele trebuie să fie exportate într-un format tabelar.

De asemenea, în loc să fie nevoie să citești sau să scrii fiecare celulă, SheetJS te ajută sa mapezi foaia de calcul într-un obiect JavaScript. Concentrarea pe transferarea datelor, dar și posibilitatea de a converti datele într-o structură JSON îi permite să fie potrivit pentru o gamă largă de aplicații.

2.2 Structura

Aplicația este construită pentru a gestiona eficient materiile prime dintr-un depozit prin definirea produselor, urmărirea tranzacțiilor de intrare sau de ieșire și administrarea utilizatorilor pe roluri. Scopul principal al acesteia este de a reduce producerea de erori operaționale prin definirea clară a obiectivului fiecărui angajat.

Proiectul este bazat pe interacțiunea tuturor părților, administratorii fiind cei care vor adăuga, șterge sau edita produse, tranzacții sau personal. Aceștia se vor autentifica în platformă cu ajutorul email-ului și parolei, care după verificarea permisiunii rolului, îi vor permite utilizatorului să acceseze pagina principală. Rolul unui administrator este de a defini catalogul de produse și de utilizatori, de a supraveghea stocul și de a corecta situațiile excepționale, dacă este cazul.

Operatorul în schimb, este persoana care primește catalogul de produse, iar cu acesta pregătește mișcările de stoc, reprezentând practic tranzacțiile de intrare și de ieșire. Nu îi este permis să adauge produse noi, iar în ceea ce privește recepționarea de tranzacții acesta are câteva limitări, iar pagina utilizatorilor nu este activă pentru acest rol.

Toate acțiunile aplicației sunt transparente, fiind mereu vizibilă persoana care a produs schimbări bazei de date a proiectului, iar claritatea interfeței impune reguli care previn scăderea stocului sub zero sau introducerea de date invalide.

2.1 Interfața cu operatorul

Interfața destinată operatorului este gândită ca fiind utilizată pentru treburi mai uzuale sau care nu necesită o responsabilitate foarte mare. Imediat după autentificare, operatorul ajunge pe pagina de start (Dashboard) unde îi este afișat numele și rolul.

Examinarea produselor se face dintr-un tabel care are detalii precum denumirea, SKU-ul, unitatea de măsură, stocul curent, stocul minim, raftul (locația fizică), prețul și autorul ultimei modificări. În bara de căutare poți să găsești produsele după nume, SKU, raft, autor și descriere, iar produsele care au un stoc sub nivelul stocului minim sunt evidențiate vizual pentru a permite utilizatorului să ia măsuri în privința aprovizionării. În meniul de acțiuni există secțiunea „Detalii produs” care permite să vizualizarea tuturor componentelor produsului, inclusiv și poza acestuia.

Înregistrarea tranzacțiilor reprezintă scopul zilnic al operatorilor. Responsabilitatea lor este de a crea tranzacții de tip „IN” (intrare, recepție) sau „OUT” (ieșire/consum/livrare) pentru un produs, selectându-i tipul, cantitatea și descrierea (care este opțională). Sistemul verifică dacă tipul este IN/OUT, cantitatea să fie un număr întreg mai mare decât zero și dacă stocul este suficient, asta în condițiile tranzacțiilor de ieșire.

Pentru a fi păstrată diferența dintre cele două roluri, operatorul are posibilități limitate de editare asupra mișcărilor de stoc deja înregistrate, modificările majore rămân astfel responsabilitatea administratorului. Atât lista de produse, cât și istoricul tranzacțiilor pot fi exportate în format Excel, respect orice filtru introdus în tabelul de valori a fiecăruia, astfel orice înregistrare sau materie primă să poate fi accesibilă oricui.

2.1 Interfața cu administratorul

Administratorul lucrează pe aceeași interfață ca operatorul, dar are la dispoziție pagini și acțiuni suplimentare acolo unde este nevoie de responsabilitate și decizie. Autentificare pentru ambele roluri rămâne aceeași, sesiunea fiind păstrată prin cookie-uri de sesiune. Paginile aferente acestui rol sunt pagina de start (Dashboard) cu numele și rolul său, paginile pentru produse și respectiv tranzacții și cea a utilizatorilor, special creată acestui rol.

Definirea catalogului de produse se realizează prin pagina corespunzătoare acestei acțiuni unde, administratorul poate crea, edita sau șterge articolele din stoc. Pentru fiecare produs trebuie completată denumirea, SKU-ul unic, unitatea de măsură, stocul și stocul minim, raftul locației fizice, descrierea, prețul și dacă se dorește și link-ul către o imagine care ajută la identificarea mai clară a acestuia.

Administratorul poate să creeze, editeze sau să șteargă orice tranzacție inclusiv schimbarea tipului acesteia, subliniind importanța unui mediu controlat asupra stocului. La salvarea sau editarea fiecărei mișcări de stoc, recalcularea materiilor prime realizându-se instant și corespunzător. Când ștergerea unui stoc sub zero are loc, operația este blocată pentru a preveni dezechilibrările.

Gestionarea utilizatorilor este atribuită doar acestui rol, dar este desfășurată cu aceeași logică și simplitate. Administratorul poate să creeze conturi noi unde trebuie introduse date precum nume, prenume, adresa de email, parola și rolul pentru cei care au nevoie de acces în aplicație, dar poate să editeze sau să elimine utilizatori dacă este cazul, fiind singura persoana care poate edita rolul altui client în aplicație.

Eficiența aplicației se bazează pe informațiile care sunt vizibile pentru fiecare dintre elementele din aceasta, fiind clar și rapid cine a făcut ultima modificare și ce produse sunt sub stocul minim. Exportul în formatul Excel permite păstrarea filtrelor active pe ecran și facilitează funcționarea optimă a platformei.

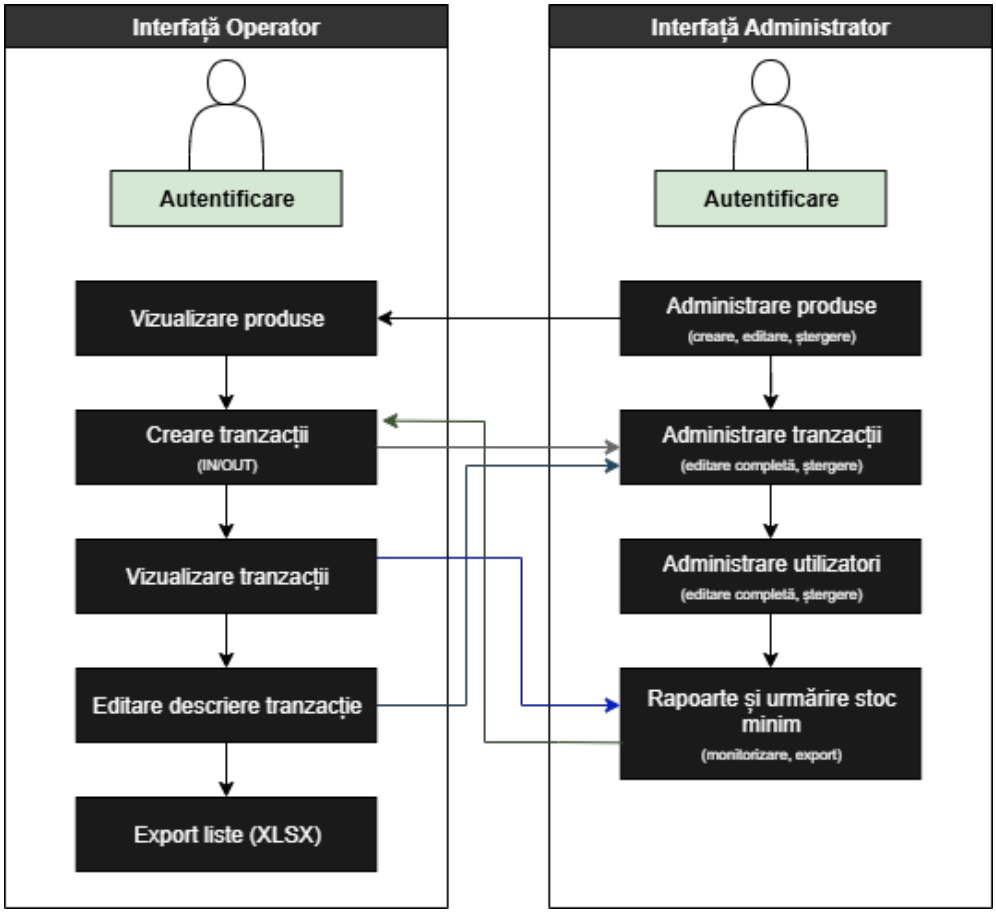


Fig.2.1 Modalitate de lucru a interfețelor

2.3 Logica aplicației

Aplicația se bazează pe cele două nivele de acces, Operator și Administrator, păstrând pentru fiecare o interfață simplă pentru gestiunea de materii prime dintr-un depozit. Acțiunile din cadrul său se realizează cu ajutorul paginilor și butoanelor aferente fiecărei nevoi ale utilizatorului. După autentificarea acestuia, în pagina principală (Dashboard) se observă numele și rolul său, iar sesiunea este păstrată cu ajutorul cookie-urilor, astfel încât reconectarea frecventă să nu fie necesară.

Principalul scop al aplicației este reprezentat de activitatea dintre produse și tranzacții. În lista de produse sunt oferite detalii precum denumirea, SKU-ul unic, unitatea de măsură, stocul curent, stocul minim, raftul (locația fizică), prețul, descrierea și autorul ultimei modificări. Detalierea oricărui produs este afișată printr-un buton „Detalii produs” unde se poate vizualiza și

imaginea acestuia, dacă aceasta a fost introdusă la crearea articolului. Dacă se dorește exportarea unui anumit produs într-un tabel Excel, acest lucru este posibil cu ajutorul meniului de căutare care funcționează multi-câmp pentru numele, SKU-ul, raftul, autorul și descrierea produsului.

Tranzacțiile reprezintă nucleul operării zilnice, iar principalul său utilizator este operatorul care selectează produsul, tipul operațiunii (IN/OUT – intrare/ieșire) și cantitatea. Dacă apar nereguli cum ar fi în cazul cantității unde trebuie introdus un număr întreg strict pozitiv sau stoc inexistent în cazul tranzacțiilor de ieșire, se vor afișa mesaje clare precum „Stoc insuficient” sau „Cantitate invalidă”. Pentru o bună funcționare a programului, fiecare tranzacție reține stocul (înainte și după fiecare operațiune), autorul și momentul exact al creării sau editării.

Gestionarea utilizatorilor este reprezentată de administrator care poate crea conturi cu nume, prenume, email, parolă și rol, totodată să modifice rolurile sau de să elimine accesul în aplicație. Supravegherea produselor care au intrat sub minimul acceptat îi revin tot administratorului care constituie responsabilitatea platformei de lucru, care trebuie să urmărească și sa cerceteze orice neregularitate care se arată pe tot parcursul desfășurării organizatorice a depozitului.

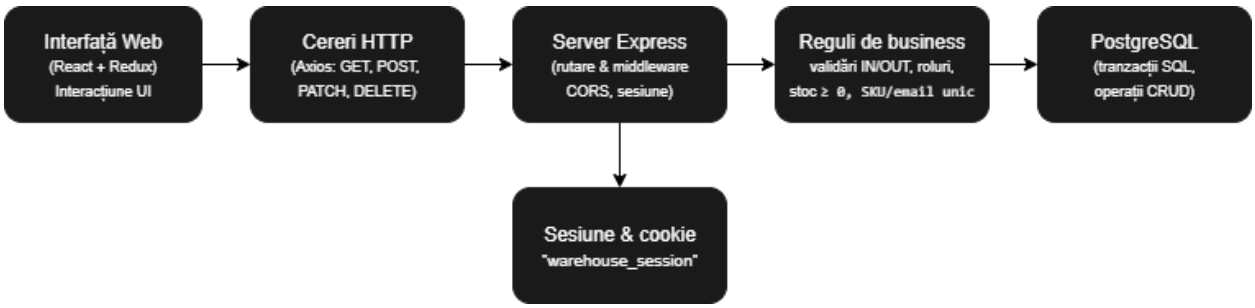


Fig.2.2 Diagrama flux a aplicației

3. PROIECTAREA DETALIATĂ A APLICAȚIEI INFORMATICE

3.1 Crearea unui proiect back-end și configurarea aplicației

Pentru realizarea acestei aplicații, a fost necesară instalarea Node.js și PostgreSQL. Serviciul back-end a fost construit cu Express și folosește pachete precum „pg” pentru accesul la baza de date, „dotenv” pentru variabilele ascunse, „bcrypt” pentru hașurarea parolelor, „express-session” pentru gestionarea sesiunii și „cors” pentru verificarea accesului dintre back-end și front-end. Toate liniile de cod necesare pentru funcționarea platformei au fost scrise în Visual Studio Code, fiind un editor gratuit, foarte popular printre programatori.

Configurarea scheletului de bază al aplicației este definit de rularea comenzii „npm init -y” care creează un fișier „package.json” cu setări implicite, de adăugarea pachetelor necesare prin comanda „npm i express pg dotenv bcrypt express-session cors” și de folosirea „npm install -g nodemon” care repornește automat aplicația prin „nodemon index.js” de fiecare dată când detectează schimbări la codul sursă.

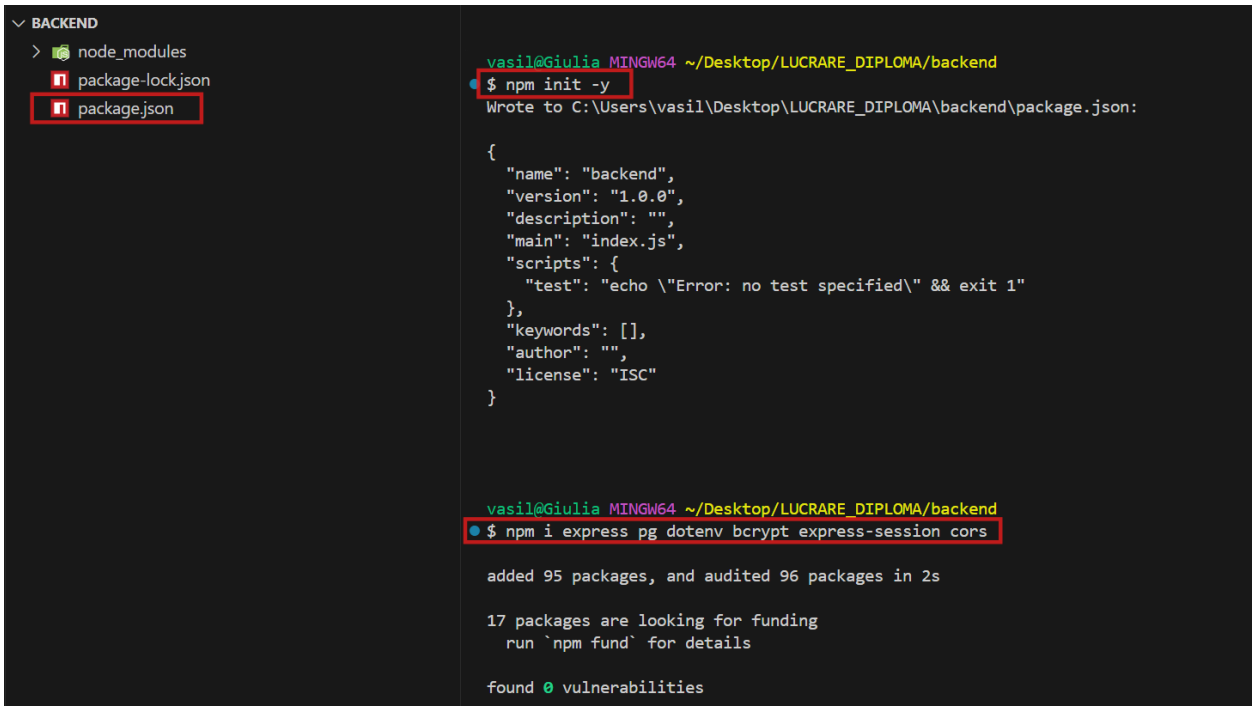


Fig.3.1 Instalarea pachetelor necesare

Conectarea la baza de date se realizează în interiorul folder-ului și fișierului „config/connectDB.js” unde, se citesc din „.env” variabilele ascunse necesare relației dintre baza de date și aplicație. După realizarea conexiunii, în consola VS Code apare mesajul „Conectat la BD!”

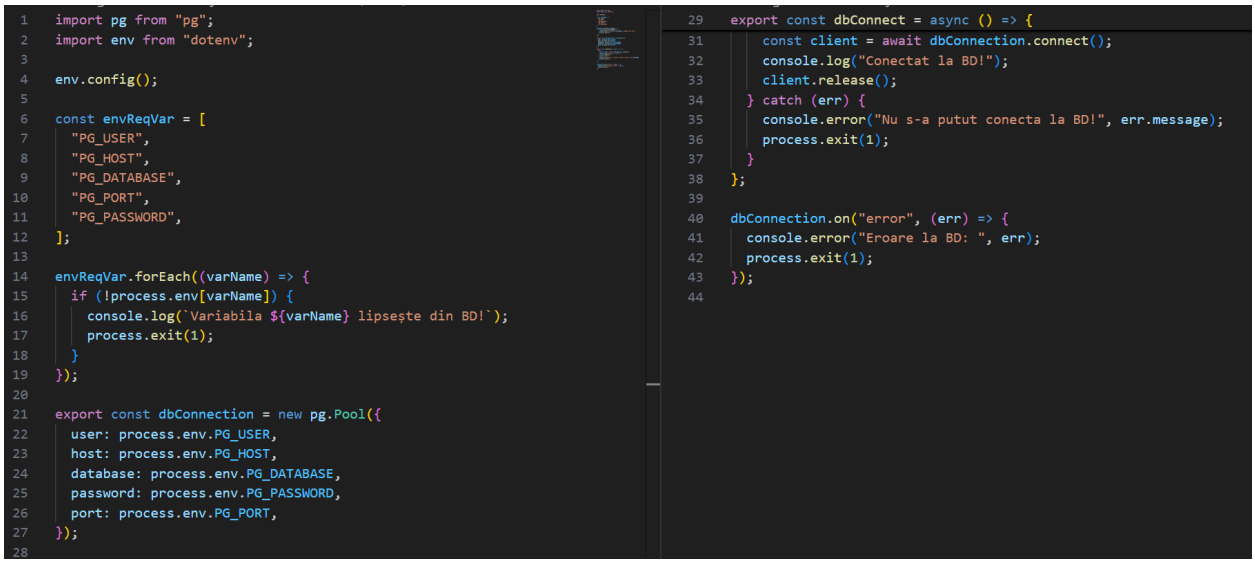
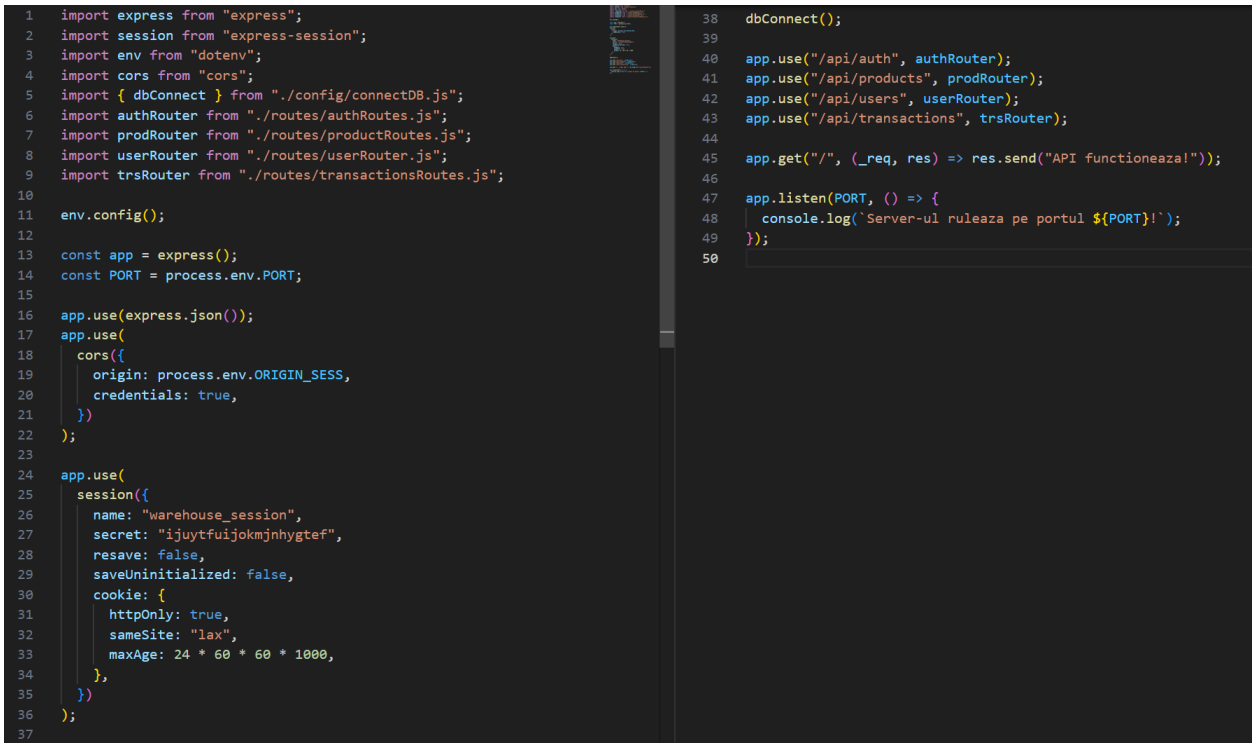


Fig.3.2 Codul necesar conectării cu PostgreSQL

Back-end-ul pornește în „index.js” prin încărcarea variabilelor ascunse din „.env”, pregătirea API-urilor cu middleware-ul „express.json()”, configurarea CORS pe originea front-end-ului și prin permiterea cookie-urilor de sesiune între domenii în timpul autentificării.

Gestiunea sesiunii se face cu ajutorul pachetului „express-session” unde cookie-ul cu numele „warehouse_session” are o durată de viață de 24 de ore și nu este re-salvat inutil. După confirmarea conexiunii, serverul montează rutele necesare, domeniile de funcționalitate fiind separate și ușor de testat. Pentru verificarea endpoint-ului se face o testare minimală unde acesta răspunde cu „API funcționează!”.



```
import express from "express";
import session from "express-session";
import env from "dotenv";
import cors from "cors";
import { dbConnect } from "./config/connectDB.js";
import authRouter from "./routes/authRoutes.js";
import prodRouter from "./routes/productRoutes.js";
import userRouter from "./routes/userRouter.js";
import trsRouter from "./routes/transactionsRoutes.js";

env.config();

const app = express();
const PORT = process.env.PORT;

app.use(express.json());
app.use(
  cors({
    origin: process.env.ORIGIN_SESS,
    credentials: true,
  })
);

app.use(
  session({
    name: "warehouse_session",
    secret: "ijuytfuijokmjnhygtef",
    resave: false,
    saveUninitialized: false,
    cookie: {
      httpOnly: true,
      sameSite: "lax",
      maxAge: 24 * 60 * 60 * 1000,
    },
  })
);

dbConnect();

app.use("/api/auth", authRouter);
app.use("/api/products", prodRouter);
app.use("/api/users", userRouter);
app.use("/api/transactions", trsRouter);

app.get("/", (_req, res) => res.send("API functioneaza!"));

app.listen(PORT, () => {
  console.log(`Server-ul ruleaza pe portul ${PORT}!`);
});
```

Fig.3.2 Codul necesar conectării la baza de date în index.js

3.2 Crearea bazei de date PostgreSQL și conectarea la server

Baza de date a fost creată în programul PostgreSQL și conține tabelele „users”, „products” și „transactions” acoperind astfel, personalul depozitului, catalogul de produse și istoricul mișcărilor de stoc. Conectarea aplicației la aceasta de realizează printr-un pool care permite reutilizarea datelor, iar verificarea este suportată de funcția „dbConnect()” care confirmă practic conexiunea.

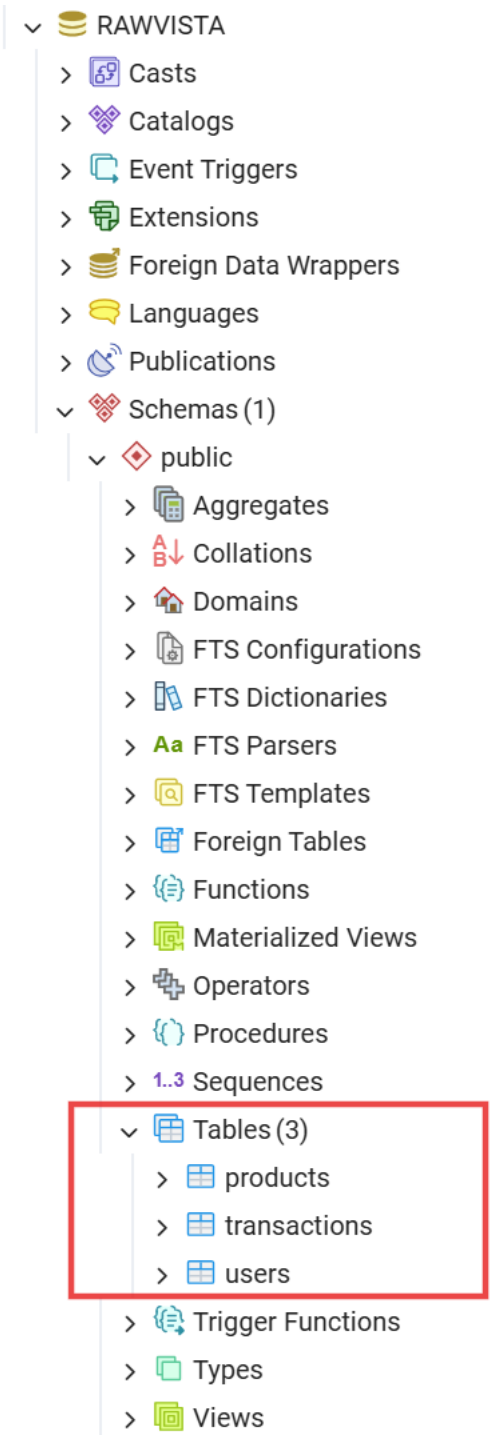


Fig.3.3 Afișarea tabelelor în PostgreSQL

Fig.3.4 IMAGINE RELATII INTRE TABELE

(transcris coloane tabele și funcțiile lor)

3.3 Modele de date și accesul la baza de date

Accesul la baza de date din PostgreSQL este format în folder-ul „models/” care reprezintă centralizarea rutelor de acces și transformarea SQL-ului într-o interfață clară pe care rutele o folosesc. Practic, cu ajutorul modelelor create pentru fiecare tabel, codul devine mai lizibil și ușor de întreținut, protejând astfel regulile codului sursă.

În „usersModel.js” sunt implementate funcțiile „createUser”, „getUserEmail”, „getUserId”, „getAllUsers”, „updateUser”, „deleteUser” care ajută la transferarea informațiilor din tabele pentru prelucrarea lor ulterioară în cadrul platformei.

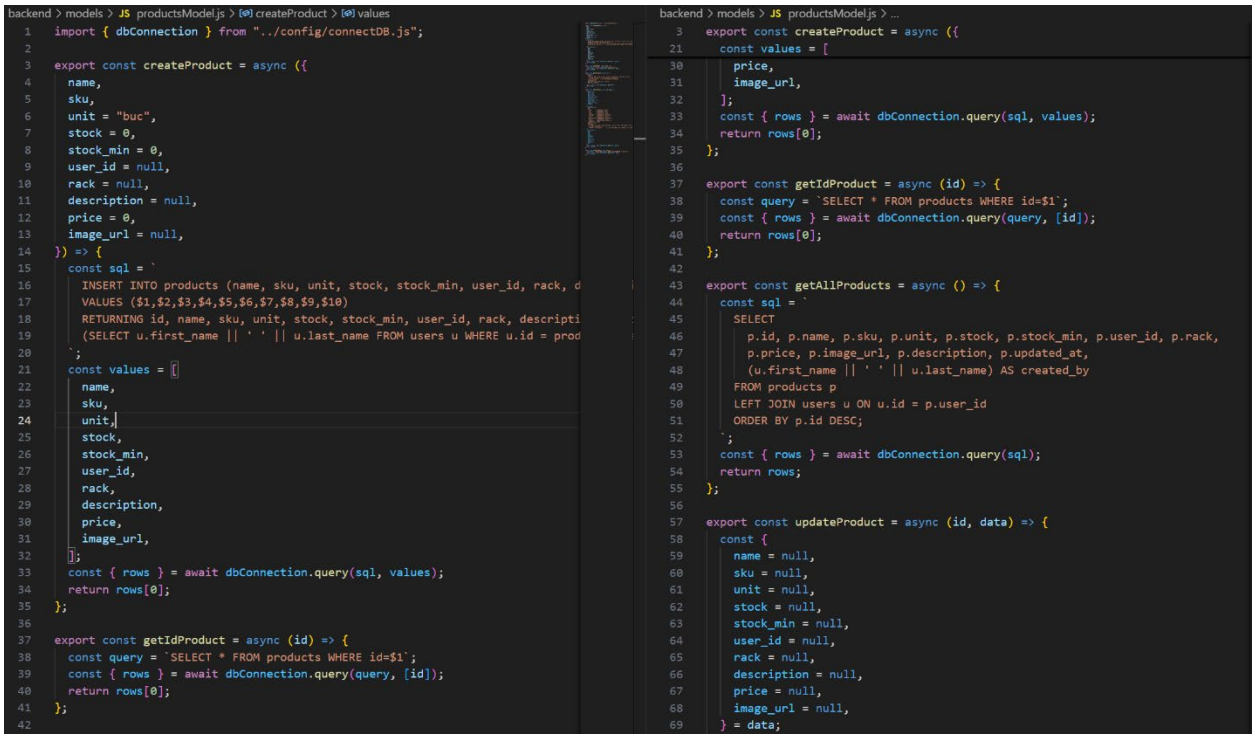
```
export const updateUser = async (id, first_name, last_name, email, role) => {
  const query = `
    UPDATE USERS
    SET first_name = COALESCE($1, first_name),
    last_name = COALESCE($2, last_name),
    email = COALESCE($3, email),
    role = COALESCE($4, role)
    WHERE id = $5
    RETURNING id, first_name, last_name, email, role;
    `;

  const values = [first_name, last_name, email, role, id];
  const { rows } = await dbConnection.query(query, values);
  return rows[0];
};
```

Fig.3.5 Funcția updateUser

Conform Fig.3.5 „updateUser” este una dintre cele mai dezvoltate funcții din „usersModel.js”, rolul său fiind de a actualiza un utilizator deja existent în tabelul „users” și de a întoarce rândul parțial actualizat. Valoarea „COALESCE” este cea care aplică o modificare parțială pentru fiecare coloană unde, dacă datele primite sunt „NULL”, se păstrează valoarea veche, orice altă valoare trimisă devenind valoarea respectivă, coloana actualizându-se.

În „productsModel.js” sunt implementate funcțiile „createProduct”, „getIdProduct”, „getAllProducts”, „updateProduct”, „deleteProduct” care ajută la gestionarea câmpurilor principale ale produsului.



```
backend > models > JS productsModel.js > createProduct > values
1   import { dbConnection } from "../config/connectDB.js";
2
3   export const createProduct = async ({
4     name,
5     sku,
6     unit = "buc",
7     stock = 0,
8     stock_min = 0,
9     user_id = null,
10    rack = null,
11    description = null,
12    price = 0,
13    image_url = null,
14  }) => {
15    const sql = `
16      INSERT INTO products (name, sku, unit, stock, stock_min, user_id, rack, d
17      VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10)
18      RETURNING id, name, sku, unit, stock, stock_min, user_id, rack, descripti
19      (SELECT u.first_name || ' ' || u.last_name FROM users u WHERE u.id = prod
20    `;
21    const values = [
22      name,
23      sku,
24      unit,
25      stock,
26      stock_min,
27      user_id,
28      rack,
29      description,
30      price,
31      image_url,
32    ];
33    const { rows } = await dbConnection.query(sql, values);
34    return rows[0];
35  };
36
37  export const getIdProduct = async (id) => {
38    const query = `SELECT * FROM products WHERE id=$1`;
39    const { rows } = await dbConnection.query(query, [id]);
40    return rows[0];
41  };
42
```

```
backend > models > JS productsModel.js > ...
3   export const createProduct = async ({
21    const values = [
30      price,
31      image_url,
32    ];
33    const { rows } = await dbConnection.query(sql, values);
34    return rows[0];
35  };
36
37  export const getIdProduct = async (id) => {
38    const query = `SELECT * FROM products WHERE id=$1`;
39    const { rows } = await dbConnection.query(query, [id]);
40    return rows[0];
41  };
42
43  export const getAllProducts = async () => {
44    const sql = `
45      SELECT
46        p.id, p.name, p.sku, p.unit, p.stock, p.stock_min, p.user_id, p.rack,
47        p.price, p.image_url, p.description, p.updated_at,
48        (u.first_name || ' ' || u.last_name) AS created_by
49      FROM products p
50      LEFT JOIN users u ON u.id = p.user_id
51      ORDER BY p.id DESC;
52    `;
53    const { rows } = await dbConnection.query(sql);
54    return rows;
55  };
56
57  export const updateProduct = async (id, data) => {
58    const {
59      name = null,
60      sku = null,
61      unit = null,
62      stock = null,
63      stock_min = null,
64      user_id = null,
65      rack = null,
66      description = null,
67      price = null,
68      image_url = null,
69    } = data;
```

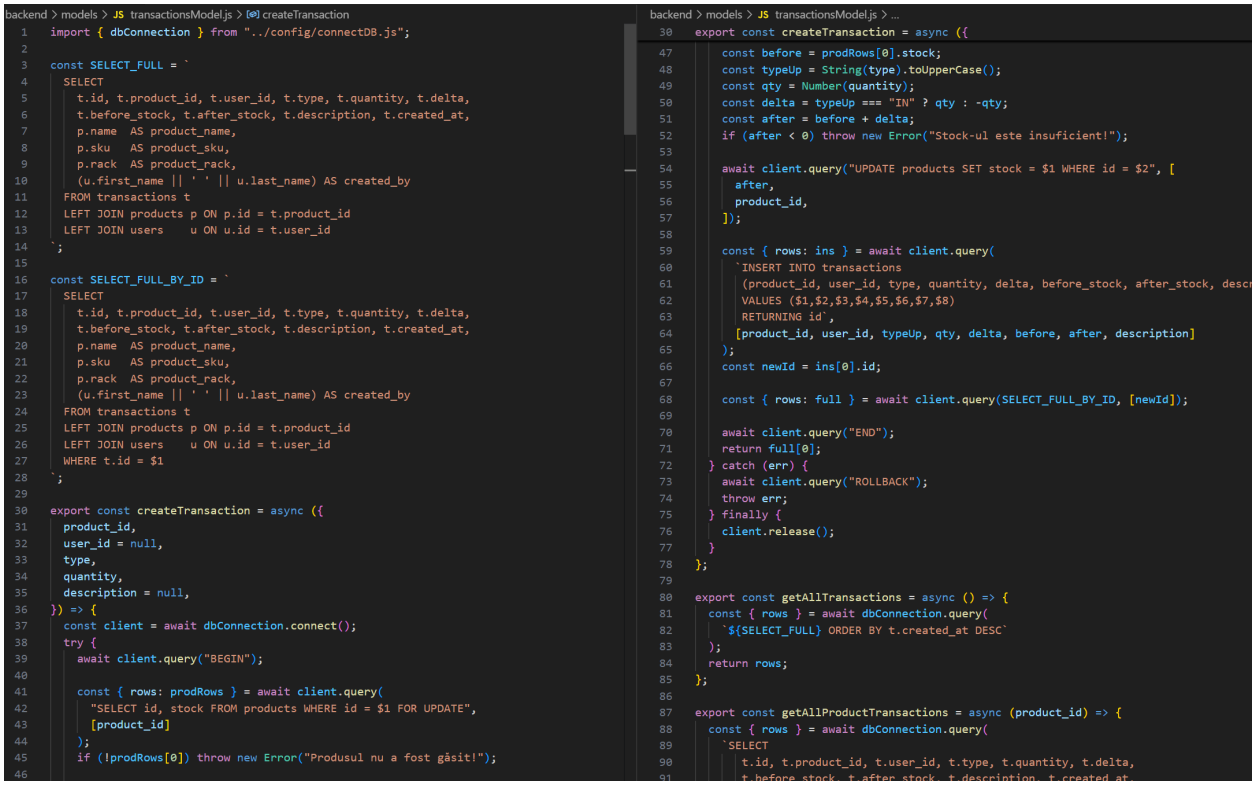
Fig.3.6 Codul modelului productsModel.js (partea 1)



```
70
71    const sql = `
72      UPDATE products
73      SET
74        name        = COALESCE($1, name),
75        sku         = COALESCE($2, sku),
76        unit        = COALESCE($3, unit),
77        stock       = COALESCE($4, stock),
78        stock_min   = COALESCE($5, stock_min),
79        user_id     = COALESCE($6, user_id),
80        rack        = COALESCE($7, rack),
81        description = COALESCE($8, description),
82        price       = COALESCE($9, price),
83        image_url   = COALESCE($10, image_url),
84        updated_at  = NOW()
85      WHERE id = $11
86      RETURNING
87        id, name, sku, unit, stock, stock_min, user_id, rack, description, price, image_url,
88        created_at, updated_at,
89        (SELECT u.first_name || ' ' || u.last_name FROM users u WHERE u.id = products.user_id) AS created_by
90    `;
91    const values = [
92      name,
93      sku,
94      unit,
95      stock,
96      stock_min,
97      user_id,
98      rack,
99      description,
100     price,
101     image_url,
102     id,
103   ];
104   const { rows } = await dbConnection.query(sql, values);
105   return rows[0];
106 };
107
108 export const deleteProduct = async (id) => {
109   const query = `DELETE FROM products WHERE id=$1 RETURNING id,name,sku`;
110   const { rows } = await dbConnection.query(query, [id]);
111   return rows[0];
112 };
```

Fig.3.7 Codul modelului productsModel.js (partea 2)

În „transactionsModel.js” fiecare mișcare de stoc păstrează o trasabilitate completă prin descrierea tipului și calcularea valorii „delta” care ne ajută să actualizăm sau nu stocul produsului. Produsul este blocat cu ajutorul funcției „SELECT ... FOR UPDATE” ceea ce ne permite verificarea „after_stock >= 0”, unde acesta este actualizat și inserat în tranzacție. La orice eșec al funcției, are loc „ROLLBACK” care practic anulează întreaga operație începută cu „BEGIN” astfel încât baza de date să revină la starea de dinainte de începerea operațiunilor, iar dacă aceasta este un succes „END” confirmă modificările.



```
backend > models > JS transactionsModel.js > createTransaction
import { dbConnection } from "../config/connectDB.js";

const SELECT_FULL = `
  SELECT
    t.id, t.product_id, t.user_id, t.type, t.quantity, t.delta,
    t.before_stock, t.after_stock, t.description, t.created_at,
    p.name  AS product_name,
    p.sku   AS product_sku,
    p.rack  AS product_rack,
    (u.first_name || ' ' || u.last_name) AS created_by
  FROM transactions t
  LEFT JOIN products p ON p.id = t.product_id
  LEFT JOIN users    u ON u.id = t.user_id
`;

const SELECT_FULL_BY_ID = `
  SELECT
    t.id, t.product_id, t.user_id, t.type, t.quantity, t.delta,
    t.before_stock, t.after_stock, t.description, t.created_at,
    p.name  AS product_name,
    p.sku   AS product_sku,
    p.rack  AS product_rack,
    (u.first_name || ' ' || u.last_name) AS created_by
  FROM transactions t
  LEFT JOIN products p ON p.id = t.product_id
  LEFT JOIN users    u ON u.id = t.user_id
  WHERE t.id = $1
`;

export const createTransaction = async ({
  product_id,
  user_id = null,
  type,
  quantity,
  description = null,
}) => {
  const client = await dbConnection.connect();
  try {
    await client.query("BEGIN");

    const { rows: prodRows } = await client.query(
      "SELECT id, stock FROM products WHERE id = $1 FOR UPDATE",
      [product_id]
    );
    if (!prodRows[0]) throw new Error("Produsul nu a fost găsit!");
```

```
backend > models > JS transactionsModel.js > ...
export const createTransaction = async ({
    const before = prodRows[0].stock;
    const typeUp = String(type).toUpperCase();
    const qty = Number(quantity);
    const delta = typeUp === "IN" ? qty : -qty;
    const after = before + delta;
    if (after < 0) throw new Error("Stock-ul este insuficient!");

    await client.query("UPDATE products SET stock = $1 WHERE id = $2", [
      after,
      product_id,
    ]);

    const { rows: ins } = await client.query(
      `INSERT INTO transactions
       (product_id, user_id, type, quantity, delta, before_stock, after_stock, descr
       VALUES ($1,$2,$3,$4,$5,$6,$7,$8)
       RETURNING id`,
      [product_id, user_id, typeUp, qty, delta, before, after, description]
    );
    const newId = ins[0].id;

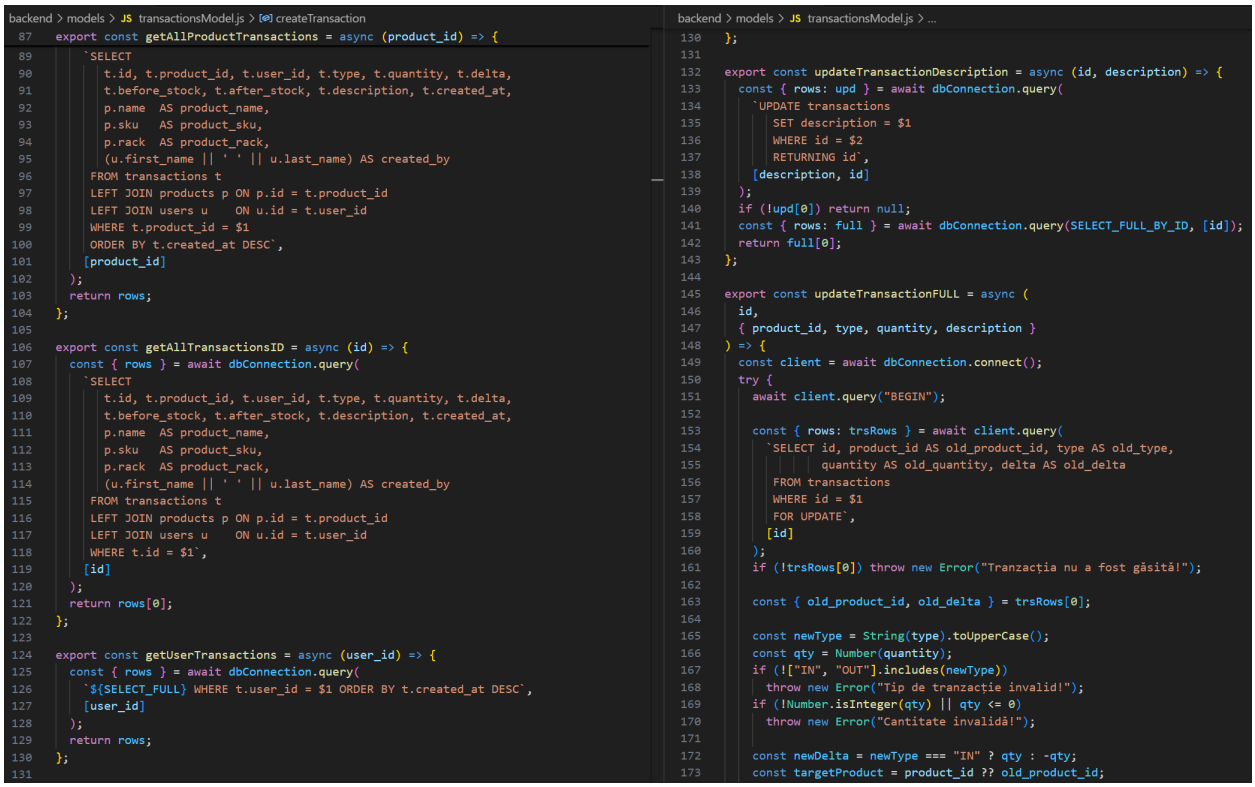
    const { rows: full } = await client.query(SELECT_FULL_BY_ID, [newId]);

    await client.query("END");
    return full[0];
  } catch (err) {
    await client.query("ROLLBACK");
    throw err;
  } finally {
    client.release();
  }
};

export const getAllTransactions = async () => {
  const { rows } = await dbConnection.query(
    `${SELECT_FULL} ORDER BY t.created_at DESC`
  );
  return rows;
};

export const getAllProductTransactions = async (product_id) => {
  const { rows } = await dbConnection.query(
    `SELECT
      t.id, t.product_id, t.user_id, t.type, t.quantity, t.delta,
```

Fig.3.8 Codul modelului transactionsModel.js (partea 1)



```
backend > models > JS transactionsModel.js > createTransaction
export const getAllProductTransactions = async (product_id) => {
    `SELECT
      t.id, t.product_id, t.user_id, t.type, t.quantity, t.delta,
      t.before_stock, t.after_stock, t.description, t.created_at,
      p.name  AS product_name,
      p.sku   AS product_sku,
      p.rack  AS product_rack,
      (u.first_name || ' ' || u.last_name) AS created_by
    FROM transactions t
    LEFT JOIN products p ON p.id = t.product_id
    LEFT JOIN users u    ON u.id = t.user_id
    WHERE t.product_id = $1
    ORDER BY t.created_at DESC`,
    [product_id]
  );
  return rows;
};

export const getAllTransactionsID = async (id) => {
  const { rows } = await dbConnection.query(
    `SELECT
      t.id, t.product_id, t.user_id, t.type, t.quantity, t.delta,
      t.before_stock, t.after_stock, t.description, t.created_at,
      p.name  AS product_name,
      p.sku   AS product_sku,
      p.rack  AS product_rack,
      (u.first_name || ' ' || u.last_name) AS created_by
    FROM transactions t
    LEFT JOIN products p ON p.id = t.product_id
    LEFT JOIN users u    ON u.id = t.user_id
    WHERE t.id = $1`,
    [id]
  );
  return rows[0];
};

export const getUserTransactions = async (user_id) => {
  const { rows } = await dbConnection.query(
    `${SELECT_FULL} WHERE t.user_id = $1 ORDER BY t.created_at DESC`,
    [user_id]
  );
  return rows;
};
```

```
backend > models > JS transactionsModel.js > ...
};

export const updateTransactionDescription = async (id, description) => {
  const { rows: upd } = await dbConnection.query(
    `UPDATE transactions
       SET description = $1
       WHERE id = $2
       RETURNING id`,
    [description, id]
  );
  if (!upd[0]) return null;
  const { rows: full } = await dbConnection.query(SELECT_FULL_BY_ID, [id]);
  return full[0];
};

export const updateTransactionFULL = async (
  id,
  { product_id, type, quantity, description }
) => {
  const client = await dbConnection.connect();
  try {
    await client.query("BEGIN");

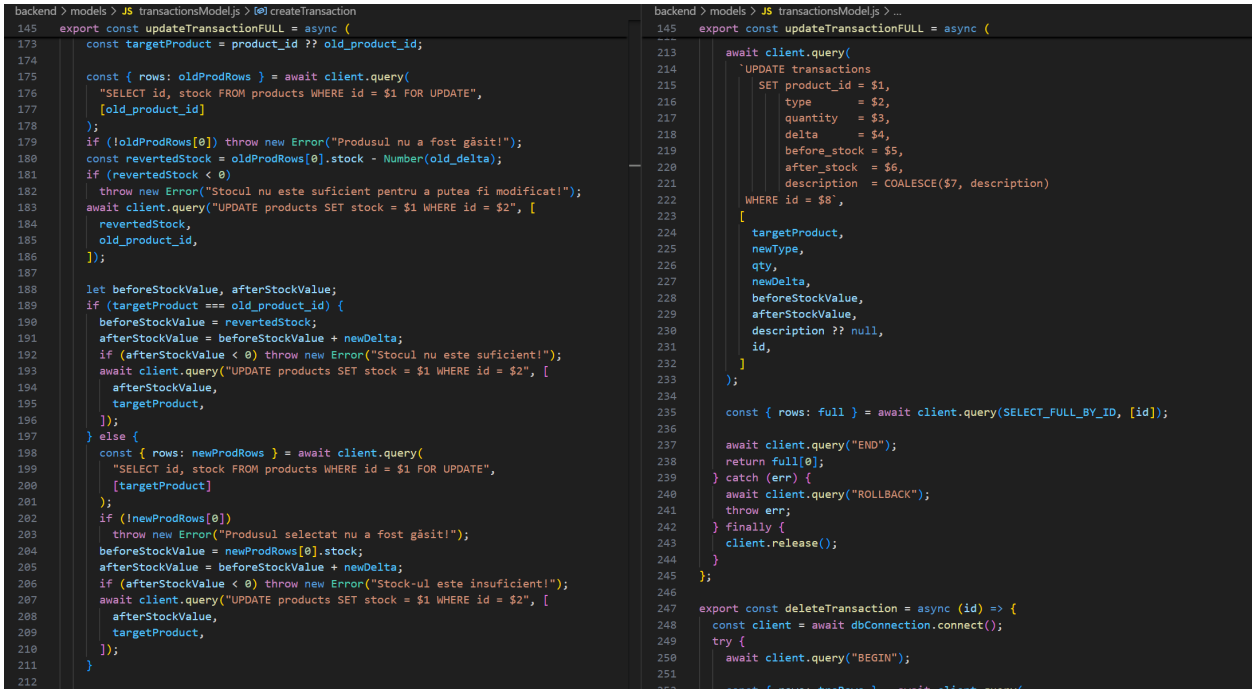
    const { rows: trsRows } = await client.query(
      `SELECT id, product_id AS old_product_id, type AS old_type,
              quantity AS old_quantity, delta AS old_delta
       FROM transactions
       WHERE id = $1
       FOR UPDATE`,
      [id]
    );
    if (!trsRows[0]) throw new Error("Tranzacția nu a fost găsită!");

    const { old_product_id, old_delta } = trsRows[0];

    const newType = String(type).toUpperCase();
    const qty = Number(quantity);
    if (!["IN", "OUT"].includes(newType))
      throw new Error("Tip de tranzacție invalid!");
    if (!Number.isInteger(qty) || qty <= 0)
      throw new Error("Cantitate invalidă!");

    const newDelta = newType === "IN" ? qty : -qty;
    const targetProduct = product_id ?? old_product_id;
```

Fig.3.9 Codul modelului transactionsModel.js (partea 2)



```
backend > models > JS transactionsModel.js > createTransaction
export const updateTransactionFULL = async (
    const targetProduct = product_id ?? old_product_id;

    const { rows: oldProdRows } = await client.query(
      "SELECT id, stock FROM products WHERE id = $1 FOR UPDATE",
      [old_product_id]
    );
    if (!oldProdRows[0]) throw new Error("Produsul nu a fost găsit!");
    const revertedStock = oldProdRows[0].stock - Number(old_delta);
    if (revertedStock < 0)
      throw new Error("Stocul nu este suficient pentru a putea fi modificat!");
    await client.query("UPDATE products SET stock = $1 WHERE id = $2", [
      revertedStock,
      old_product_id,
    ]);

    let beforeStockValue, afterStockValue;
    if (targetProduct === old_product_id) {
      beforeStockValue = revertedStock;
      afterStockValue = beforeStockValue + newDelta;
      if (afterStockValue < 0) throw new Error("Stocul nu este suficient!");
      await client.query("UPDATE products SET stock = $1 WHERE id = $2", [
        afterStockValue,
        targetProduct,
      ]);
    } else {
      const { rows: newProdRows } = await client.query(
        "SELECT id, stock FROM products WHERE id = $1 FOR UPDATE",
        [targetProduct]
      );
      if (!newProdRows[0])
        throw new Error("Produsul selectat nu a fost găsit!");
      beforeStockValue = newProdRows[0].stock;
      afterStockValue = beforeStockValue + newDelta;
      if (afterStockValue < 0) throw new Error("Stock-ul este insuficient!");
      await client.query("UPDATE products SET stock = $1 WHERE id = $2", [
        afterStockValue,
        targetProduct,
      ]);
    }
```

```
backend > models > JS transactionsModel.js > ...
export const updateTransactionFULL = async (
    await client.query(
      `UPDATE transactions
         SET product_id = $1,
             type       = $2,
             quantity   = $3,
             delta      = $4,
             before_stock = $5,
             after_stock  = $6,
             description  = COALESCE($7, description)
       WHERE id = $8`,
      [
        targetProduct,
        newType,
        qty,
        newDelta,
        beforeStockValue,
        afterStockValue,
        description ?? null,
        id,
      ]
    );

    const { rows: full } = await client.query(SELECT_FULL_BY_ID, [id]);

    await client.query("END");
    return full[0];
  } catch (err) {
    await client.query("ROLLBACK");
    throw err;
  } finally {
    client.release();
  }
};

export const deleteTransaction = async (id) => {
  const client = await dbConnection.connect();
  try {
    await client.query("BEGIN");
```

Fig.3.10 Codul modelului transactionsModel.js (partea 3)

```
246
247  export const deleteTransaction = async (id) => {
248    const client = await dbConnection.connect();
249    try {
250      await client.query("BEGIN");
251
252      const { rows: trsRows } = await client.query(
253        "SELECT id, product_id, delta FROM transactions WHERE id = $1 FOR UPDATE",
254        [id]
255      );
256      if (!trsRows[0]) throw new Error("Tranzacția nu a fost găsită!");
257
258      const { product_id, delta } = trsRows[0];
259
260      const { rows: prodRows } = await client.query(
261        "SELECT id, stock FROM products WHERE id = $1 FOR UPDATE",
262        [product_id]
263      );
264      if (!prodRows[0]) throw new Error("Produsul nu a fost găsit!");
265
266      const currentStock = prodRows[0].stock;
267      const newStock = currentStock - Number(delta);
268      if (newStock < 0)
269        throw new Error("Nu este stock suficient pentru a termina operația!");
270
271      await client.query("UPDATE products SET stock = $1 WHERE id = $2", [
272        newStock,
273        product_id,
274      ]);
275
276      const { rows: del } = await client.query(
277        "DELETE FROM transactions WHERE id = $1 RETURNING id, product_id",
278        [id]
279      );
280
281      await client.query("END");
282      return del[0];
283    } catch (err) {
284      await client.query("ROLLBACK");
285      throw err;
286    } finally {
287      client.release();
288    }
289  };
290
```

Fig.3.10 Codul modelului transactionsModel.js (partea 4)

acestuia.

Înregistrarea tranzacțiilor reprezintă s

BIBLIOGRAFIE

[1] Site oficial InvenTree: https://inventree.org, accesat 06.09.2025

[2] Site oficial Odoo: https://www.odoo.com, accesat 06.09.2025

[3] Site oficial documentație JavaScript: https://developer.mozilla.org/en-US/docs/Web/JavaScript, accesat 07.09.2025

[4] Site oficial documentație Node.js: https://nodejs.org/en/learn/getting-started/introduction-to-nodejs , accesat 07.09.2025

[5] Site oficial documentație Express: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction, accesat 08.09.2025

[6] Site oficial documentație pg: https://node-postgres.com, accesat 08.09.2025

[7] Site oficial documentație HTML: https://developer.mozilla.org/en-US/docs/Web/HTML, accesat 08.09.2025

[8] Introducere HTML: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content accesat 08.09.2025

[9] Site oficial documentație CSS: developer.mozilla.org/en-US/docs/Learn/CSS/First_steps/What_is_CSS accesat 08.09.2025

[10] Site oficial documentație Bootstrap: https://getbootstrap.com/docs/5.3/getting-started/introduction accesat 08.09.2025

[11] Site oficial documentație React: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Frameworks_libraries/Introduction , accesat 08.09.2025

[12] Introducere React: https://react.dev/learn/describing-the-ui#, accesat 08.09.2025

[13] Site oficial Vite: https://vite.dev/guide/why, accesat 08.09.2025

[14] Site introducere în Axios: https://www.geeksforgeeks.org/html/what-is-axios, accesat în 08.09.2025

[15] Site introducere în SheetJS: https://www.berg-software.com/en/get-browser-data-into-excel-with-sheetjs, 08.09.2025